

2D Graphics Algorithms

Recap - 1

- We digitally model the world of interest using geometric primitives.
- For rendering this world, we must compute the part of the world that is visible within a pixel. And then compute the pixel colour such that it visually represents that part of the world as best as it can.
- Edges (geometrically approximated by straight lines) dominate visual form perception by humans.
- How do we render edges in our digitally created world?

Point Classification

- Point classification is a very fundamental operation in computer graphics. It is defined as follows:
 - *Given a point P and a geometric entity E , classify the point as being **outside**, **on** or **inside** the entity.*
- Example:
 - $P = (x_0, y_0)$
 - Let E be a line with equation: $ax+by+c=0$
 - Let $v = ax_0+by_0+c$.
 - Then P is inside if $v<0$, on if $v=0$ and outside if $v>0$

Rasterization

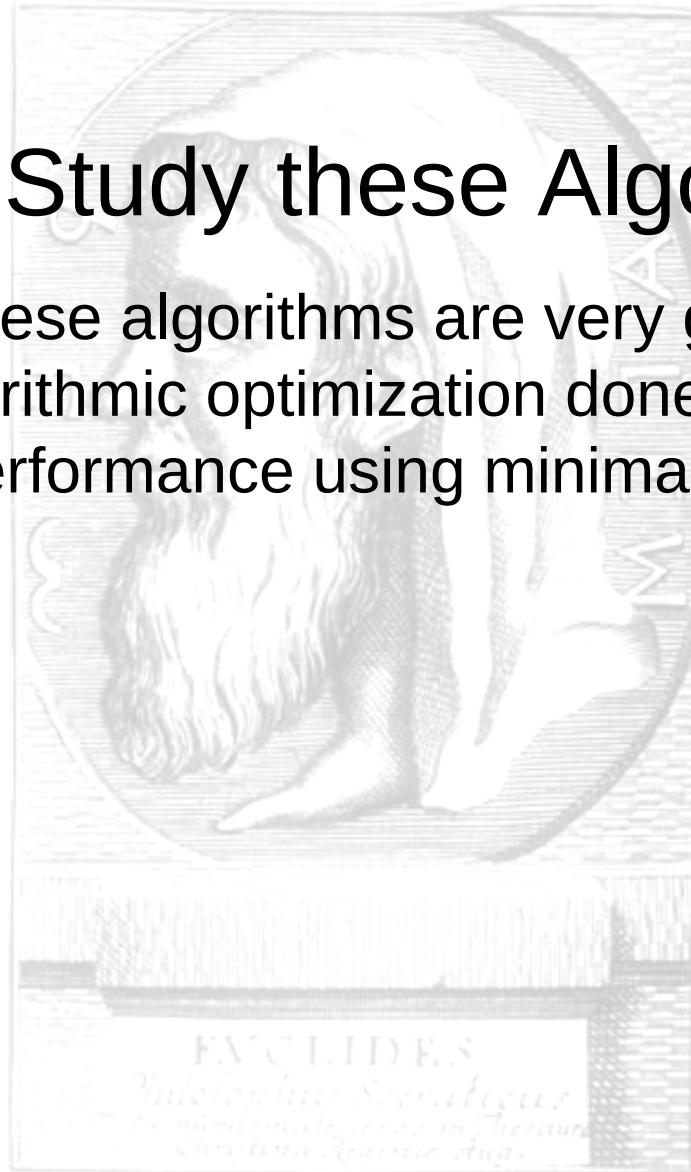
- The task of displaying a world modelled using primitives like lines, polygons, filled/patterned areas, etc. can be carried out in two steps
 - determine the pixels through which the primitive is visible, a process called ***Rasterization or scan conversion***
 - determine the color value to be assigned to each such pixel.

Raster Graphics Packages

- The efficiency of these steps forms the main criteria to determine the performance of a display
- The raster graphics package is typically a collection of efficient algorithms for ***scan converting*** (rasterization) of the display primitives
- High performance graphics workstations have most of these algorithms implemented in hardware

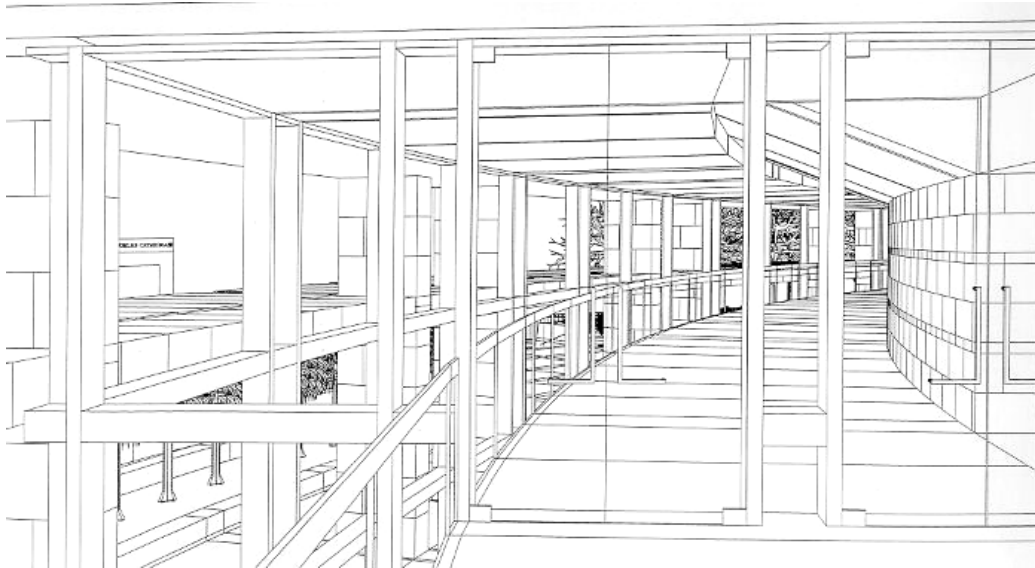
Why Study these Algorithms?

- Some of these algorithms are very good examples of clever algorithmic optimization done to dramatically improve performance using minimal hardware facilities.



Scan Converting a Line Segment

- The line is a powerful element used since the days of Euclid to model the edges in the world.



- Given a line segment defined by its endpoints determine the pixels and colour which best model the line segment.

Scan converting lines

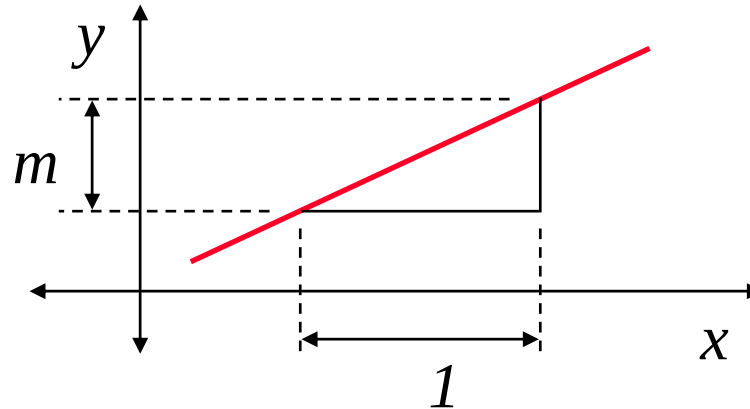
- Requirements
 - the chosen pixels should lie as close to the ideal line as possible
 - the sequence of pixels should be as straight as possible. All lines should appear to be of constant brightness independent of their length and orientation
 - should start and end accurately
 - should be drawn as rapidly as possible
 - should be possible to draw lines with different width and line styles

Equation of a Line

- Equation of a line is $y - m.x + c = 0$
- For a line segment joining points $P(x_1, y_1)$ and $P(x_2, y_2)$

$$\text{slope } m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{\Delta y}{\Delta x}$$

- Slope m means that for every unit increment in X the increment in Y is m units



Digital Differential Analyzer (DDA)

- We consider the line in the first octant. Other cases can be easily derived.
- Uses differential equation of the line

$$y_i = m \cdot x_i + c$$

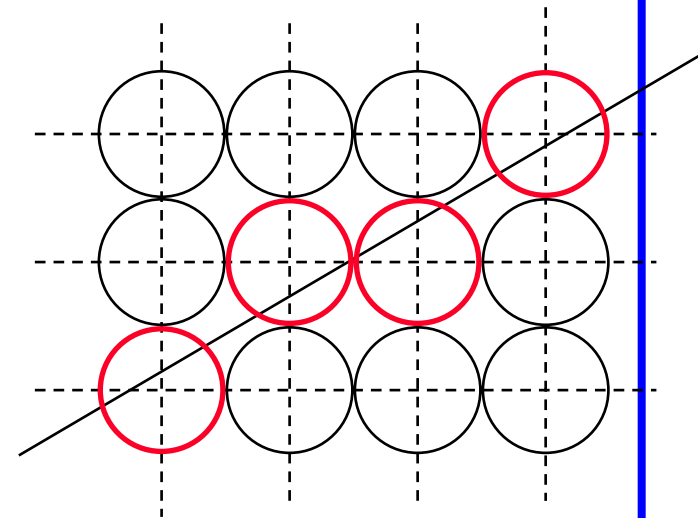
$$\text{where, } m = \frac{y_2 - y_1}{x_2 - x_1}$$

- Incrementing X-coordinate by 1

$$x_i = x_{i_prev} + 1$$

$$y_i = y_{i_prev} + m$$

- Illuminate the pixel $[x_i, \text{round}(y_i)]$

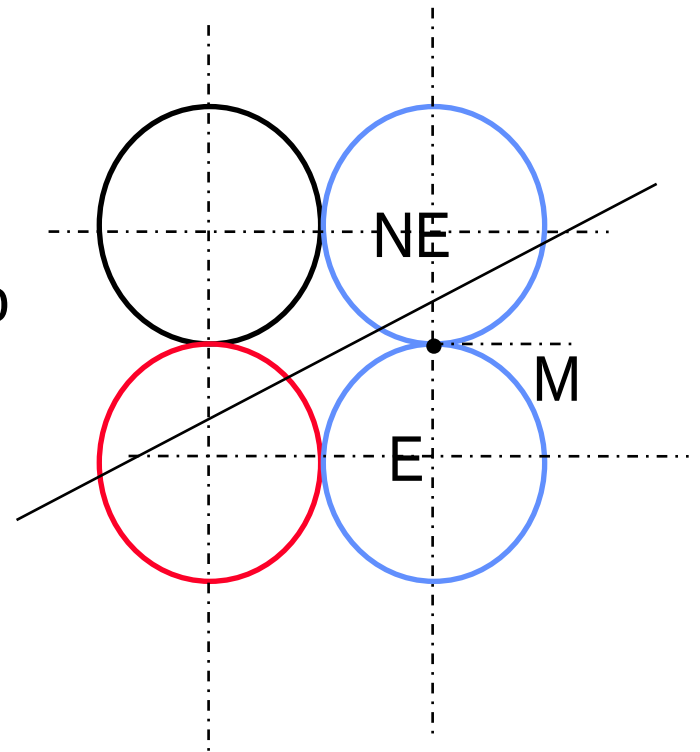


Digital Differential Analyzer

- Digital Differential Analyzer algorithm more popularly known as *DDA*
- This is an *Incremental algorithm* i.e. at each step it makes incremental calculations based on the calculations done during the preceding step
- The algorithm uses floating point operations, which are very cleverly avoided in an algorithm first proposed by J. Bresenham of IBM,. The algorithm is well known as *Bresenham's Line Drawing Algorithm*.
- We will discuss a slight variation – Midpoint Line Drawing Algorithm.

Midpoint Line Drawing

- The same incremental method for scan converting lines can be derived using an integer formulation. In this the mid-point between the East (E) and NorthEast (NE) pixels is checked to see on which side of the line it lies. For this instead of $y = mx + c$, the line equation of the form
- $F(x) = (ax + by + c = 0)$ is used.
- **Principle:** If $F(\text{mid-point})$ is below (≤ 0) the line then NE is chosen, otherwise E is chosen.



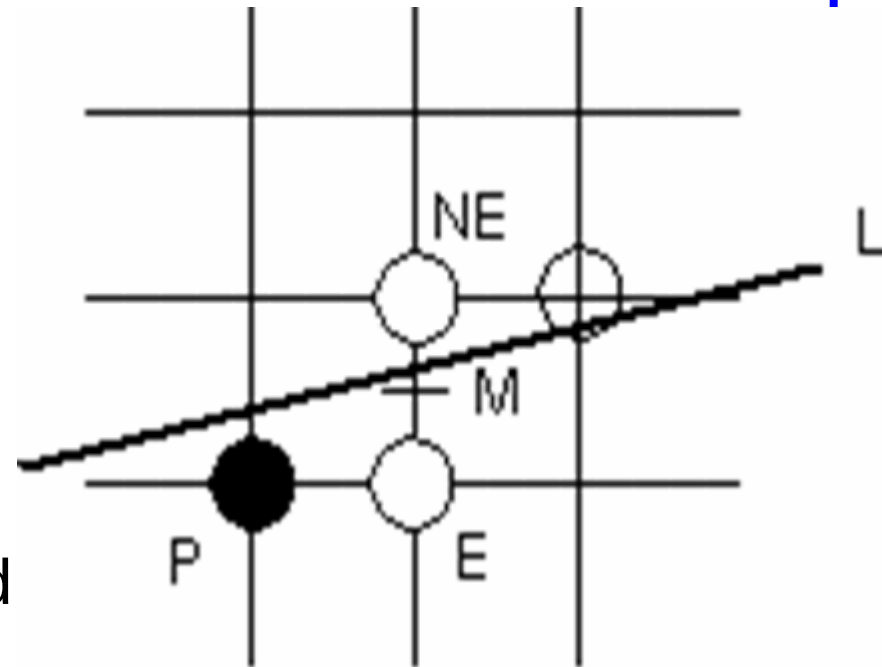
Midpoint Line Algorithm

$d = a(x_p+1) + b(y_p+1/2) + c$
is the decision variable.

if $d > 0$ then choose E else if
 $d \leq 0$ then choose NE.

For an incremental algorithm,
we must compute d
incrementally. For that, let us
see what happens to M and d
for the next grid line.

We have two cases –
old choice was E or NE



Midpoint Line Algorithm - contd

If the old choice is E, then

$$d_{\text{new}} = a(x_p + 2) + b(y_p + 1/2) + c$$

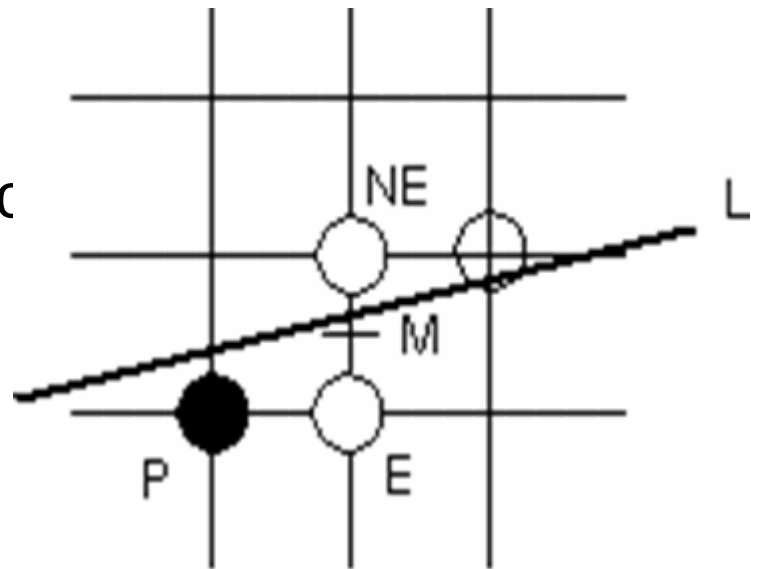
$$\text{But } d_{\text{old}} = a(x_p + 1) + b(y_p + 1/2) + c$$

$$\text{Hence } d_{\text{new}} = d_{\text{old}} + a.$$

If the old choice is NE, then

$$d_{\text{new}} = a(x_p + 2) + b(y_p + 3/2) + c$$

$$\text{Now } d_{\text{new}} = d_{\text{old}} + a + b.$$



Midpoint Line Algorithm - contd

Consider the line segment from (x_1, y_1) to (x_2, y_2) .

$(y_2 - y_1)x - (x_2 - x_1)y + c = 0$ is the equation.

$$a = dy = (y_2 - y_1), b = -dx = -(x_2 - x_1)$$

And $a + b = dy - dx$

What should d be to start with?

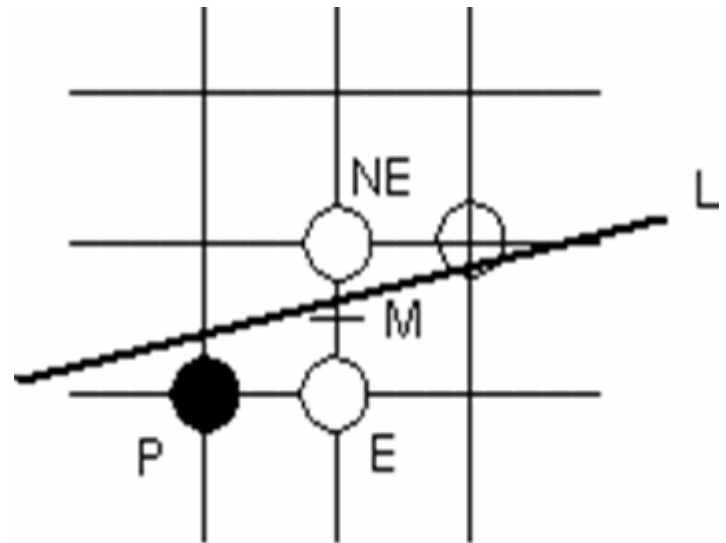
The first midpoint $M_1 = (x_1+1, y_1+1/2)$

$$\begin{aligned} F(M_1) &= d_1 = ax_1 + by_1 + c + a + b/2 \\ &= F(x_1, y_1) + a + b/2 \end{aligned}$$

x_1, y_1 is on the line, so $F(x_1, y_1) = 0$

Thus $d_1 = a + b/2$

In order to avoid division by 2, we choose to make our decision using $2d_1 = 2a + b$, which does not change sign of d .



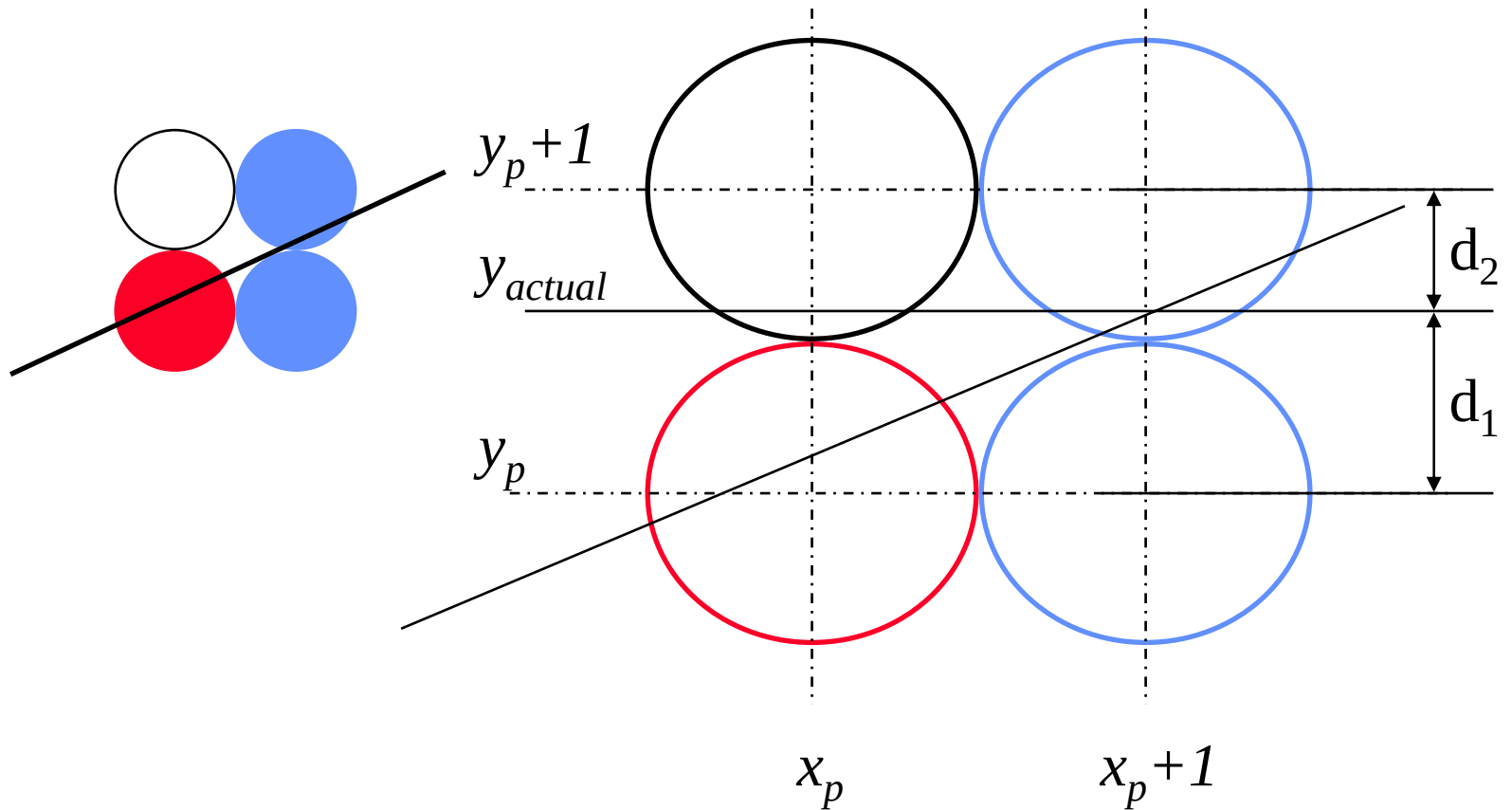
Midpoint Line Algorithm

- Input line end points $(x1, y1), (x2, y2)$
- Set $x = x1$ and $y = y1$ and $\text{SetLineColour}(x,y)$
- Calculate $dX = x2 - x1$ and $dY = y2 - y1$
- Calculate $\text{incrNE} = 2*(dY - dX)$ and $\text{incrE} = 2*dY$
- Calculate $d = 2*dY - dX$
- While ($x < x1$)
 - { $x = x + 1$;
 - If $d > 0$ then $y = y + 1$ and $d = d + \text{incrNE}$ else $d = d + \text{incrE}$;
 - $\text{SetLineColour}(x,y)$
 - }

Advantages of Incremental Midpoint Line Algorithm

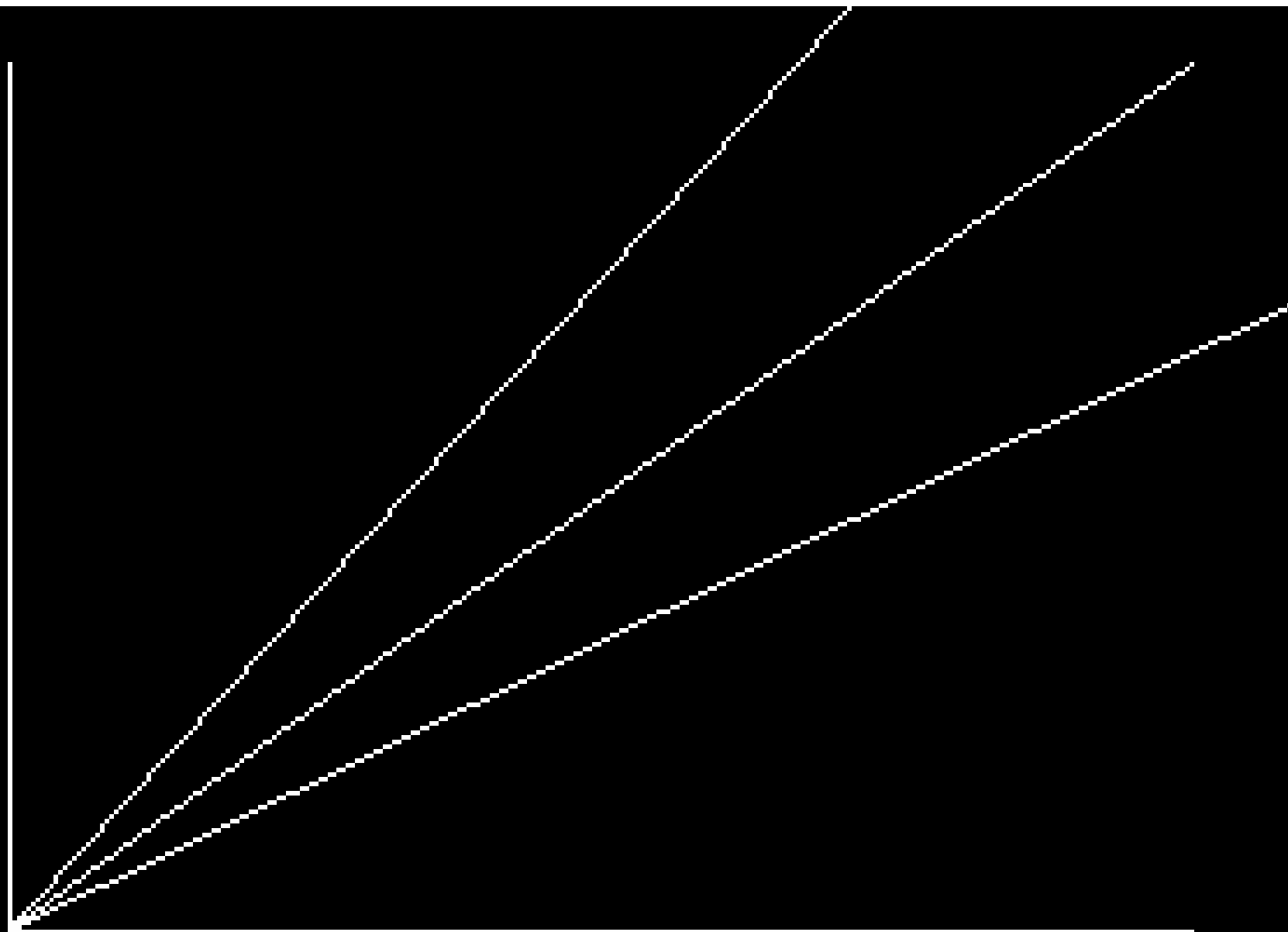
- It is an incremental algorithm
- It uses only integer arithmetic
- Provides the best fit approximation to the actual line

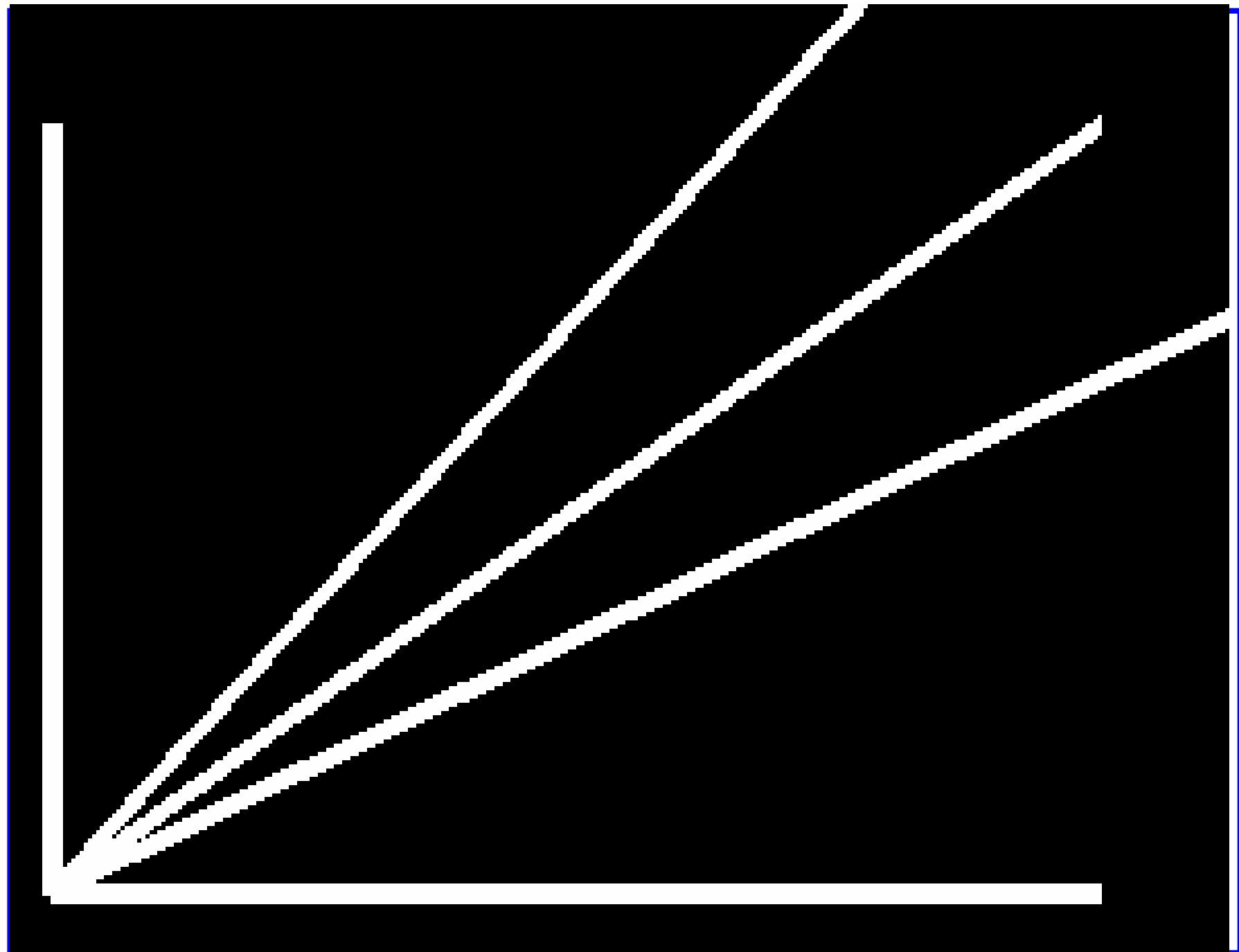
Bresenham Line Drawing

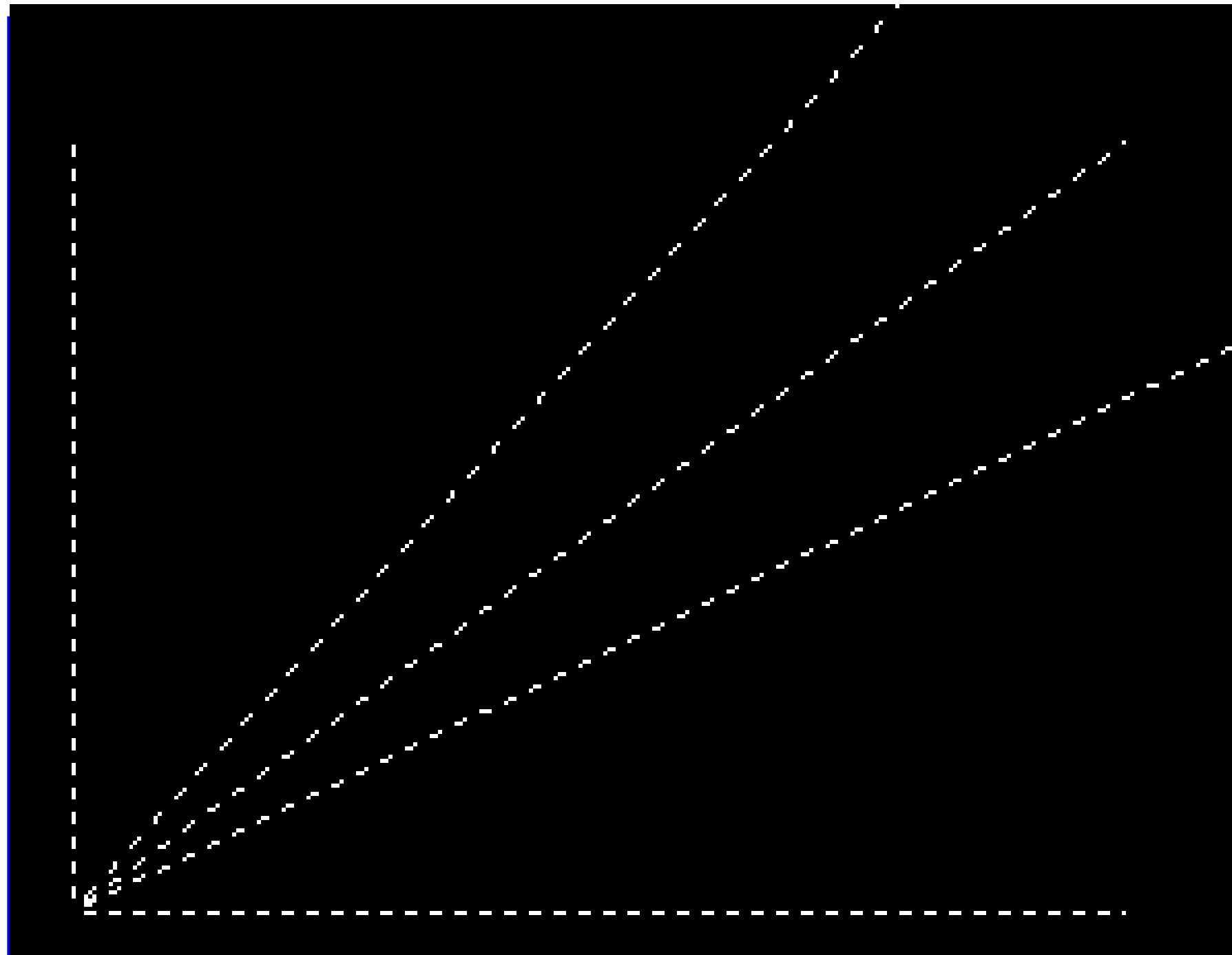


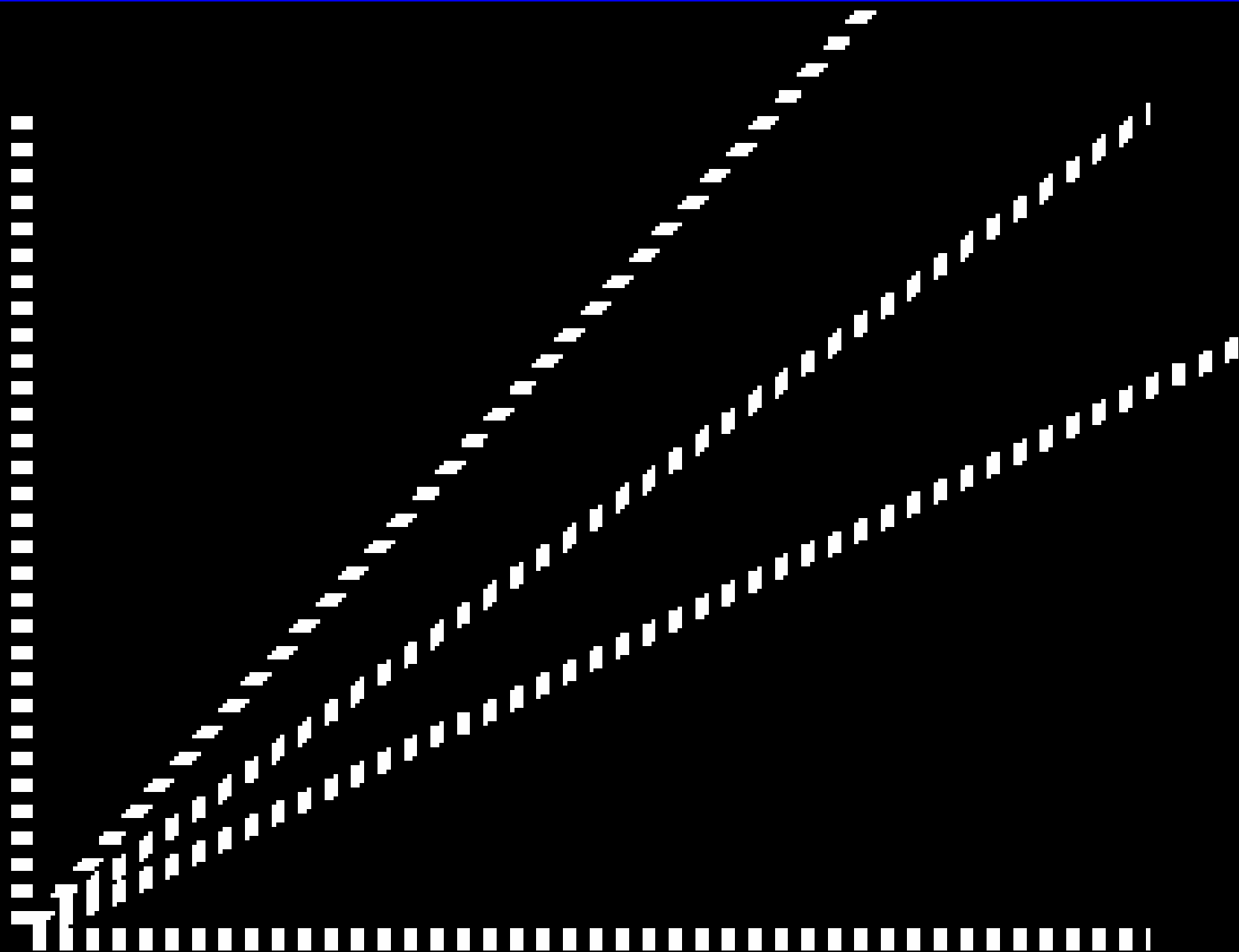
Some Additional Issues

- How do we handle line segments in other octants?
- Drawing should be endpoint order independent:
P0 - P1 should be identical to P1-P0
- If line segment goes outside the raster window, it has to be clipped; then start is no longer at the centre of any pixel. d_0 has to be suitably computed.
- Variation of intensity as function of slope – (cannot be avoided without anti-aliasing)
- Connected line segments. Common end points may be drawn multiple times. Could give rise to problems some times.

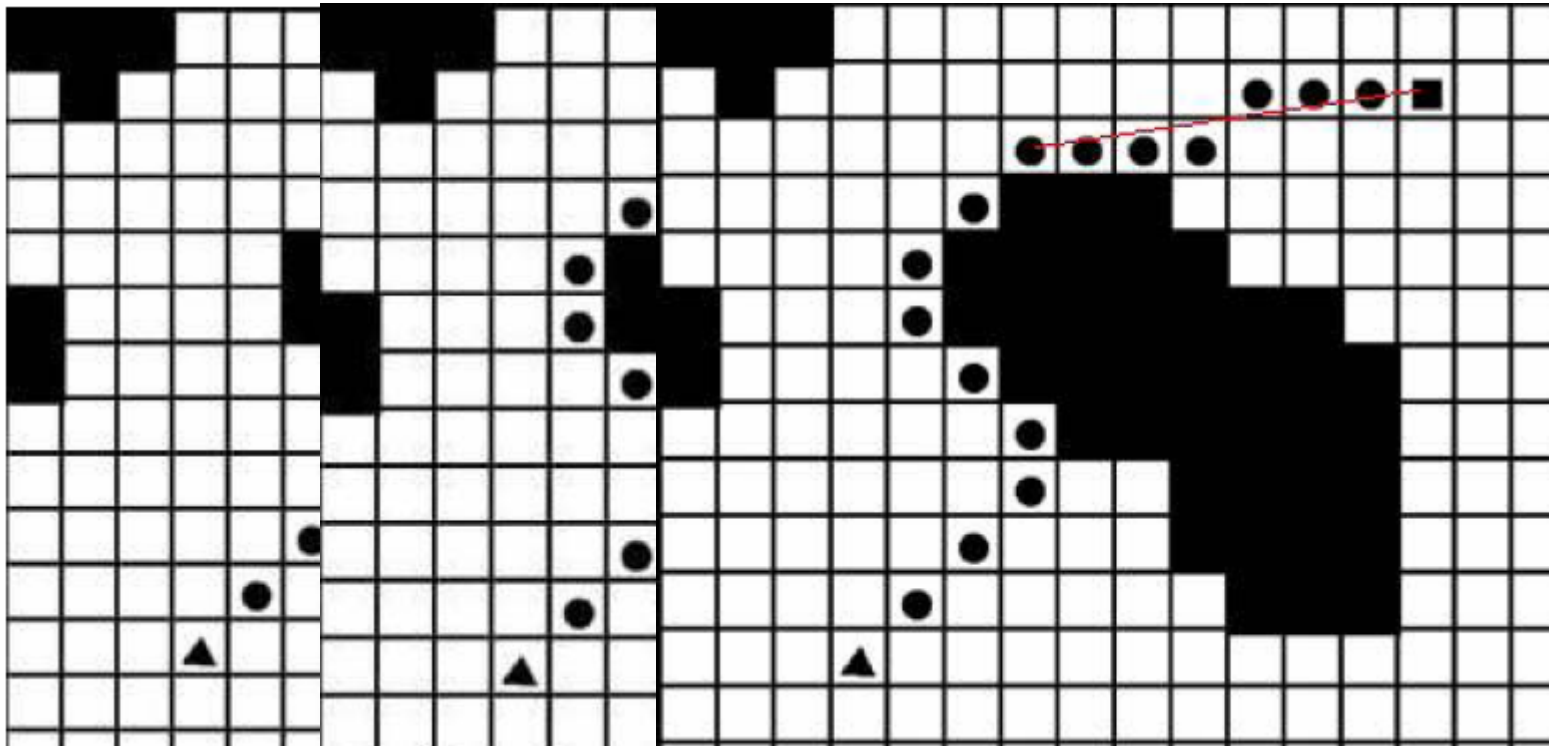




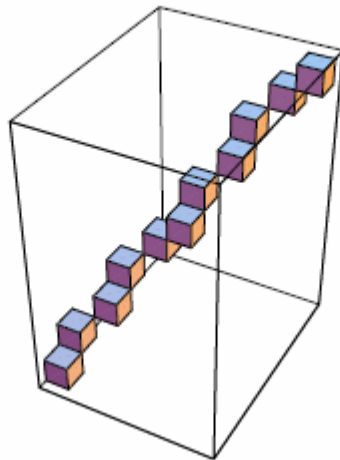
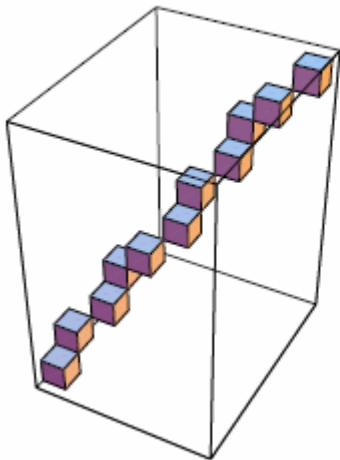
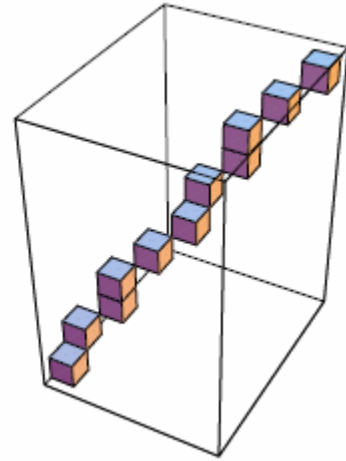
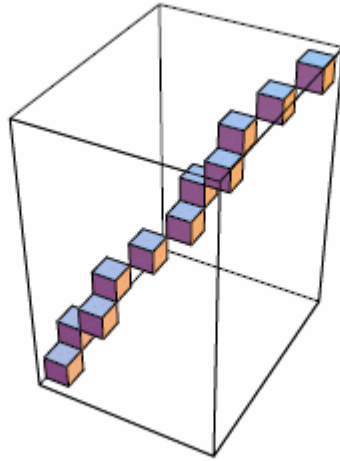
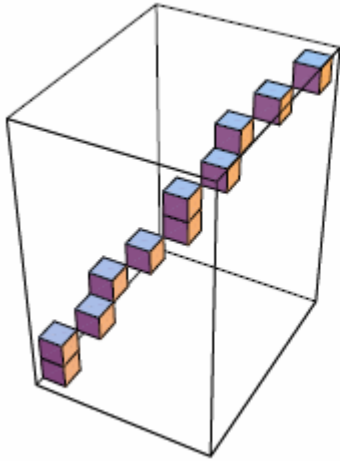




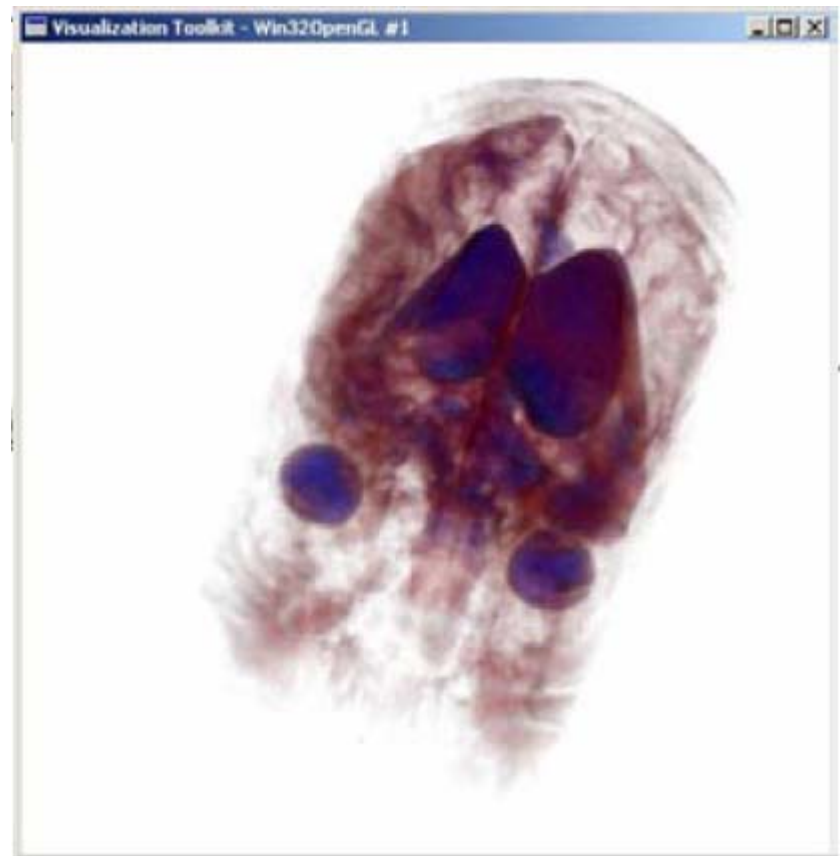
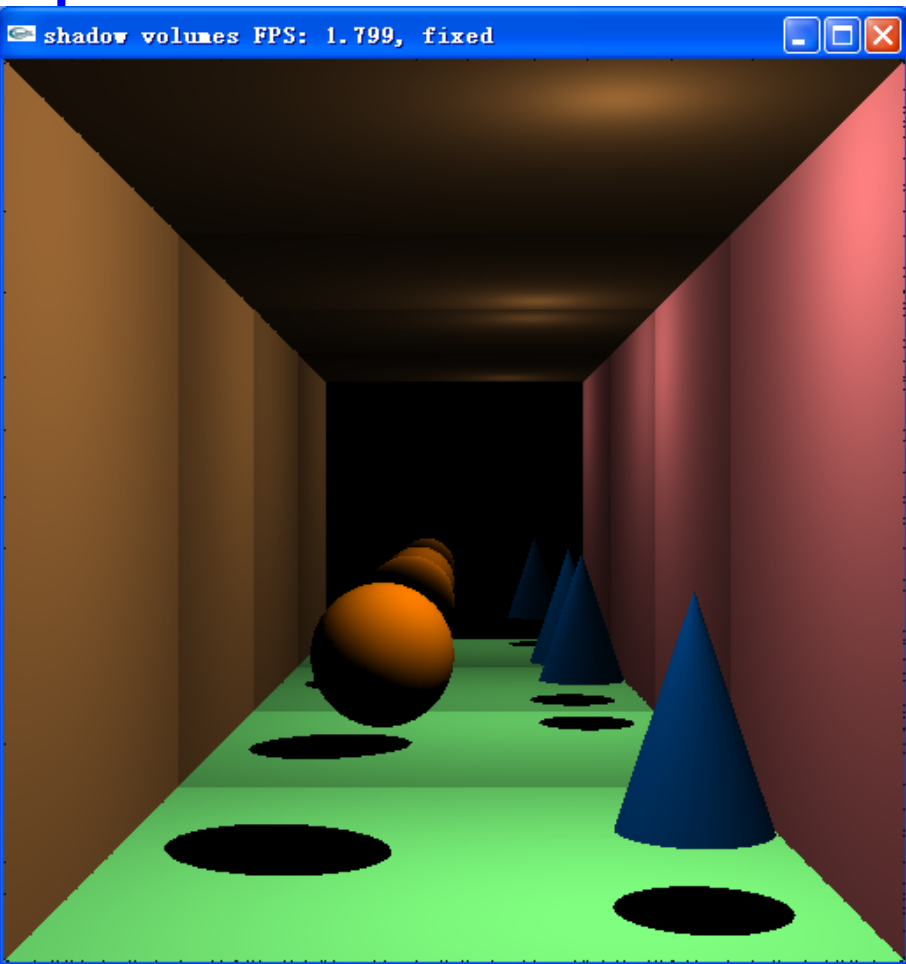
Bresenham: Path Finding



3D Bresenham



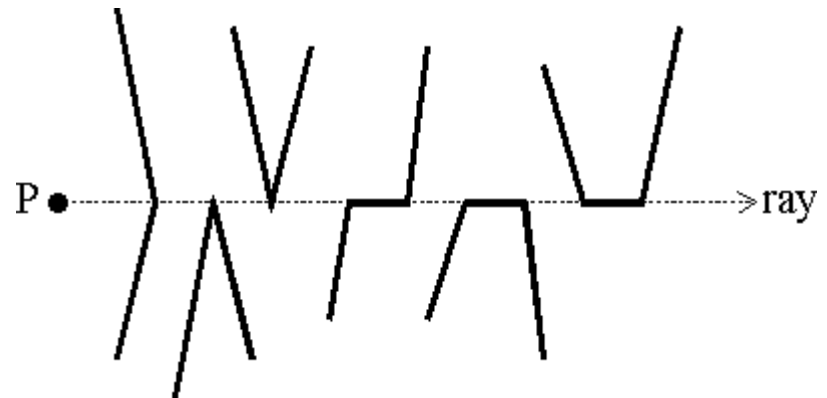
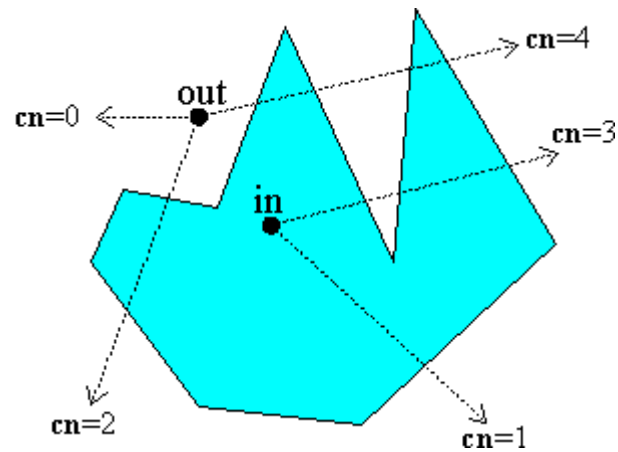
3D Bresenham



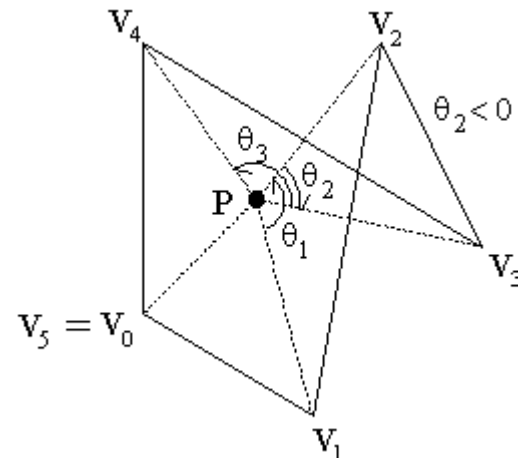
Polygon filling

- Simplest method to fill a polygonal area is to test every pixel in the raster to see if it lies inside the polygon.
- There are two methods to make an inside check
 - even-odd test
 - winding number test
- Bounding boxes can be used to improve performance

Inside check



$$\begin{aligned} \mathbf{wn} &= \frac{1}{2\pi} \sum_{i=0}^{n-1} \theta_i \\ &= \frac{1}{2\pi} \sum_{i=0}^{n-1} \arccos \left(\frac{\mathbf{PV}_i \cdot \mathbf{PV}_{i+1}}{|\mathbf{PV}_i| |\mathbf{PV}_{i+1}|} \right) \end{aligned}$$

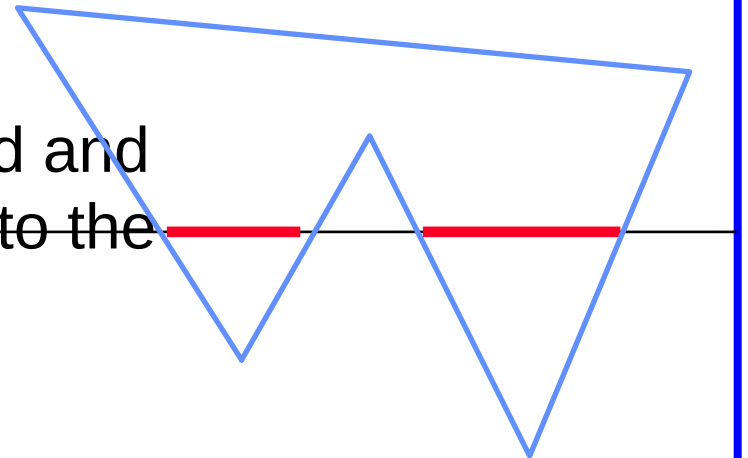


Scan Line Methods

- Makes use of the *coherence* properties
 - Spatial coherence : Except at the boundary edges, adjacent pixels are likely to have the same characteristics
 - Span coherence : Pixels in a scan line will be set to same values for solid shaded primitives
 - Scan line coherence : Pixels in the adjacent scan lines are likely to have the same characteristics
- Uses intersections between area boundaries and scan lines to identify pixels that are inside the area

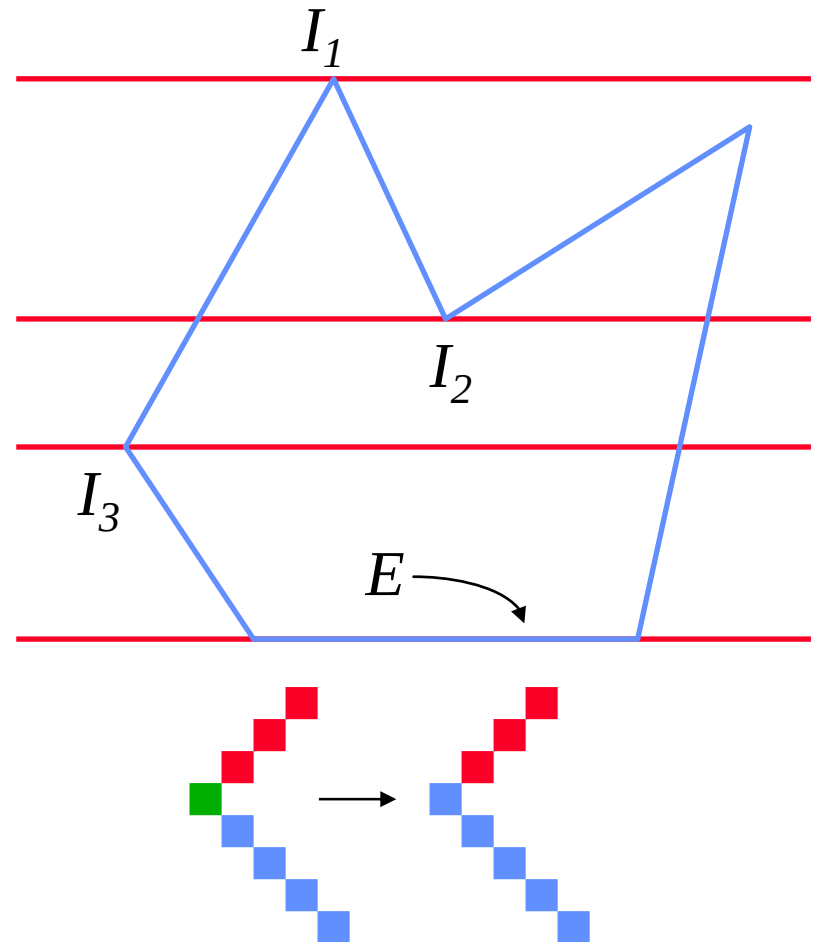
Scan Line Method

- Proceeding from left to right the intersections are paired and intervening pixels are set to the specified intensity
- Algorithm
 - Find the intersections of the scan line with all the edges in the polygon
 - Sort the intersections by increasing X-coordinates
 - Fill the pixels between pair of intersections



Special cases for Scan Line Method

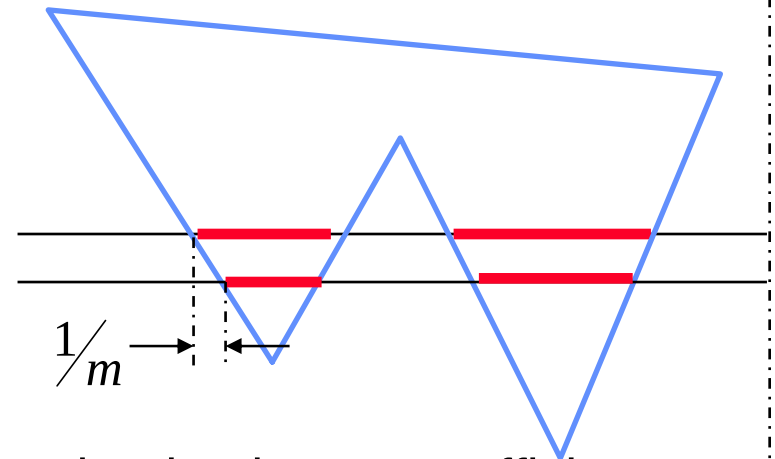
- Overall topology should be considered for intersection at the vertices
- Intersections like I_1 and I_2 should be considered as two intersections
- Intersections like I_3 should be considered as one intersection
- Horizontal edges like E need not be considered



Efficiency Issues in Scan Line Method

- Intersections could be found using edge coherence
the X-intersection value x_{i+1} of the lower scan line can be computed from the X-intersection value x_i of the preceeding scanline as

$$x_{i+1} = x_i + 1/m$$



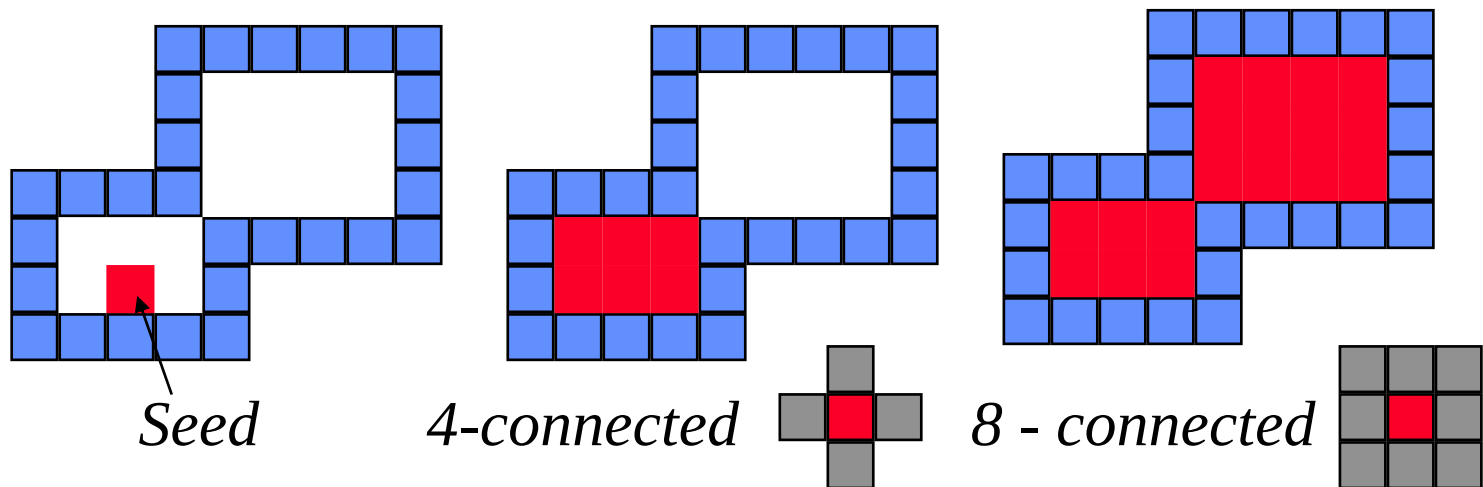
- List of active edges could be maintained to increase efficiency
- Efficiency could be further improved if polygons are convex, much better if they are only triangles

Advantages of Scan Line method

- The algorithm is efficient
- Each pixel is visited only once
- Shading algorithms could be easily integrated with this method to obtain shaded area

Seed Fill Algorithms

- Assumes that atleast one pixel interior to the polygon is known
- It is a recursive algorithm
- Useful in interactive paint packages



Aliasing

- Aliasing is caused due to the discrete nature of the display device
- Rasterizing primitives is like sampling a continuous signal i.e. representing a signal by a finite set of values (point sampling)
- Information is lost if the rate of sampling is not sufficient. This sampling error is called ***aliasing***.
- Effects of aliasing are
 - Jagged edges
 - Incorrectly rendered fine details
 - Small objects might miss

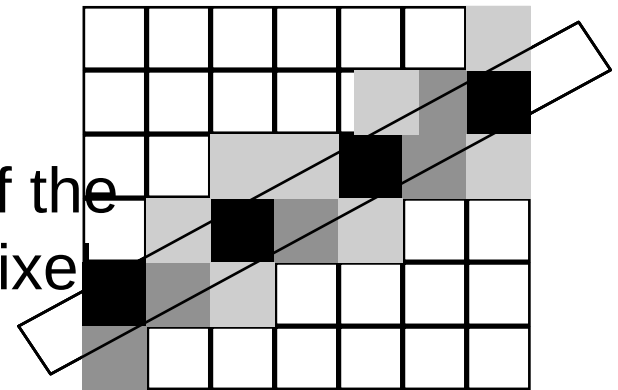


Antialiasing

- Application of techniques to reduce/eliminate aliasing artifacts
- Some of the methods are
 - increasing sampling rate by increasing the resolution. Display memory requirements increases four times if the resolution is doubled
 - averaging methods (post processing). Intensity of a pixel is set as the weighted average of its own intensity and the intensity of the surrounding pixels
 - Area sampling, more popular

Area Sampling

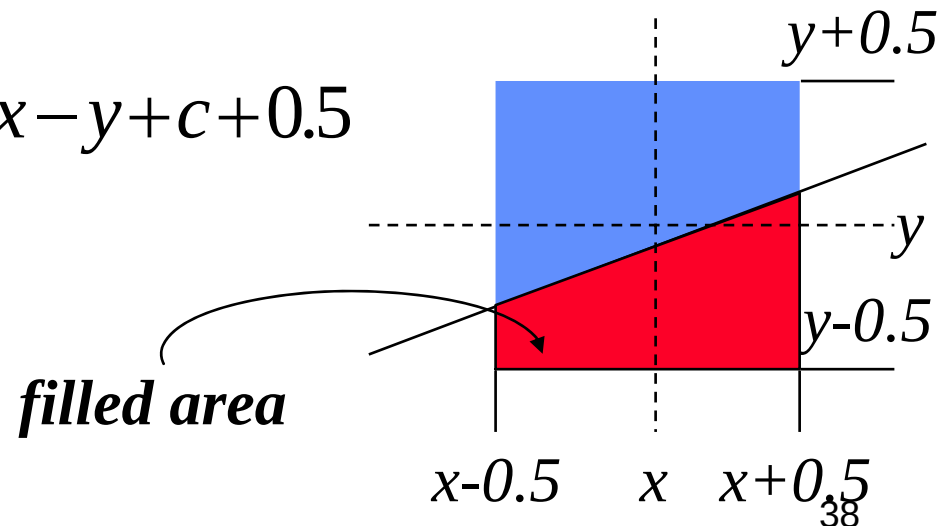
- A scan converted primitive occupies finite area on the screen
- Intensity of the boundary pixels is adjusted depending on the percent of the pixel area covered by the primitive. This is called weighted area sampling
- Intensities can be weighted depending on the distance of the area from the center of the pixel



Area Sampling

- Methods to estimate percent of pixel covered by the pixel
 - subdivide pixel into sub-pixels and determine how many sub-pixels are inside the boundary
 - Incremental line algorithm can be extended, with area calculated as

$$Area = m \cdot x - y + c + 0.5$$

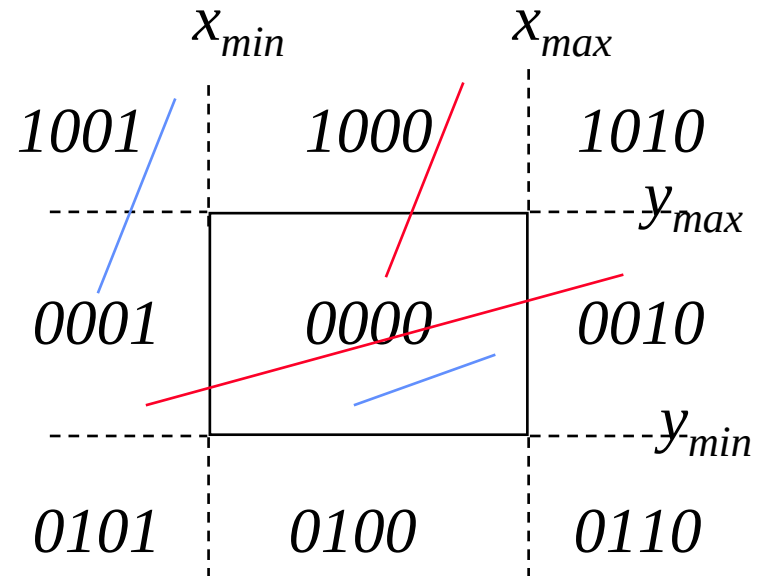


Clipping

- Clipping of primitives is done usually before scan converting the primitives
- Reasons being
 - scan conversion needs to deal only with the clipped version of the primitive, which might be much smaller than its unclipped version
 - Primitives are usually defined in the real world, and their mapping from the real to the integer domain of the display might result in the overflowing of the integer values resulting in unnecessary artifacts

Cohen-Sutherland Line-Clipping Algorithm

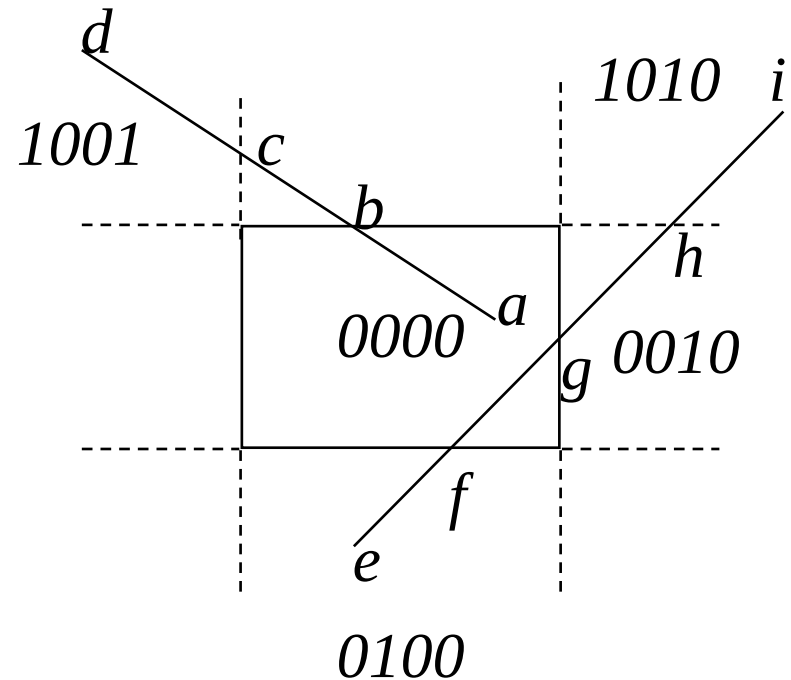
- Extend the edges of the clip rectangle to divide its plane into nine regions
- Each region is assigned a 4-bit code, determined by where the region lies w.r.t. the outside half-planes of the clip-rectangle edges
- Trivial rejection –
If the logical **and** of the 4-bit codes of the end points is non-zero, the reject.



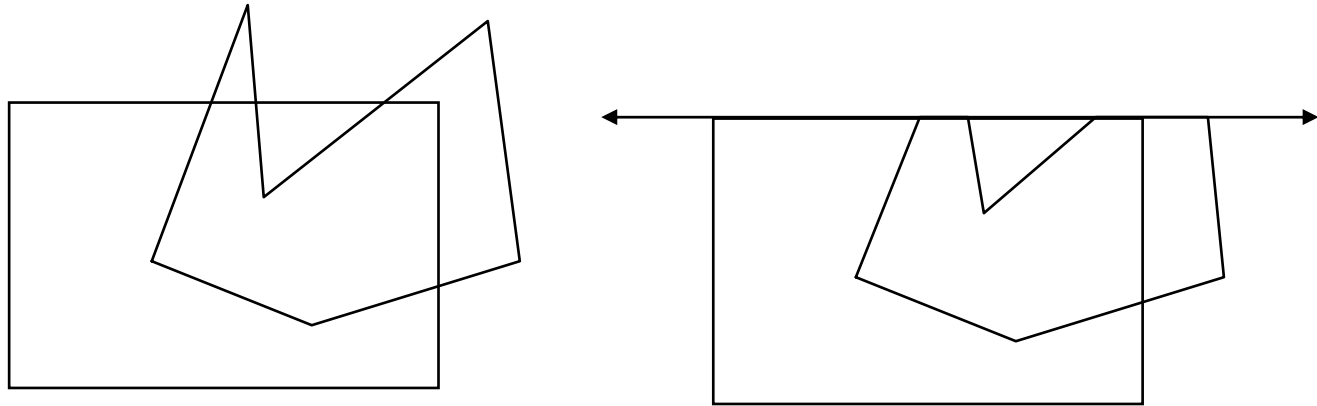
bit-1 $y > y_{max}$, bit-2 $y < y_{min}$
 bit-3 $x > x_{max}$, bit-4 $x < x_{min}$

Cohen-Sutherland Line-Clipping Algorithm

- To do the clipping find the end point that lies outside
- Test the outcode to find the edge that is crossed and determine the corresponding intersection point
- Replace the outside end-point by intersection-point
- Repeat the above steps for the new line

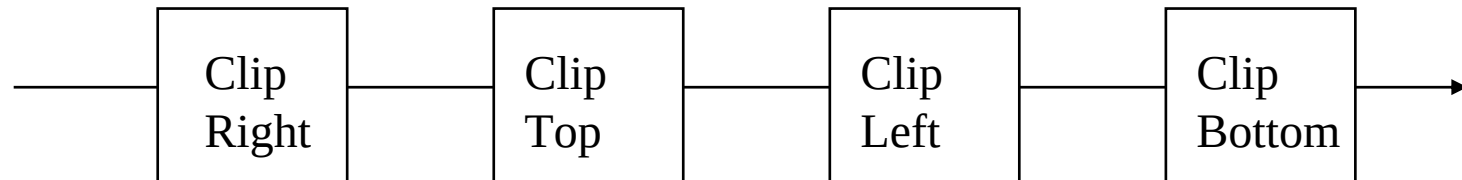


Sutherland-Hodgeman Polygon-Clipping Algorithm



- Polygons can be clipped against each edge of the window one edge at a time. Window/edge intersections, if any, are easy to find since the X or Y coordinates are already known.
- Vertices which are kept after clipping against one window edge are saved for clipping against the remaining edges.

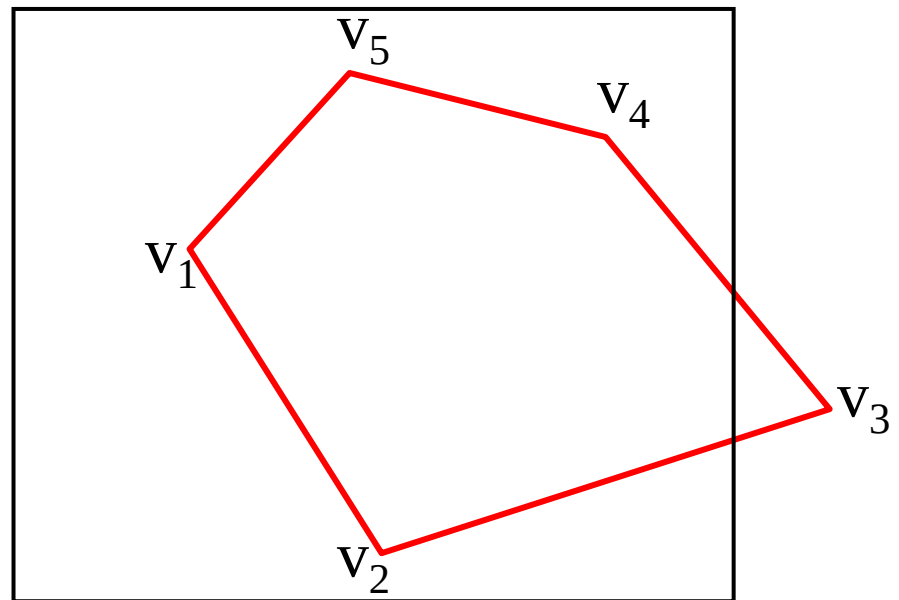
Pipelined Polygon Clipping



- Because polygon clipping does not depend on any other polygons, it is possible to arrange the clipping stages in a **pipeline**. the input polygon is clipped against one edge and any points that are kept are passed on as input to the next *stage* of the pipeline.
- This way four polygons can be at different *stages* of the clipping process simultaneously. This is often implemented in hardware.

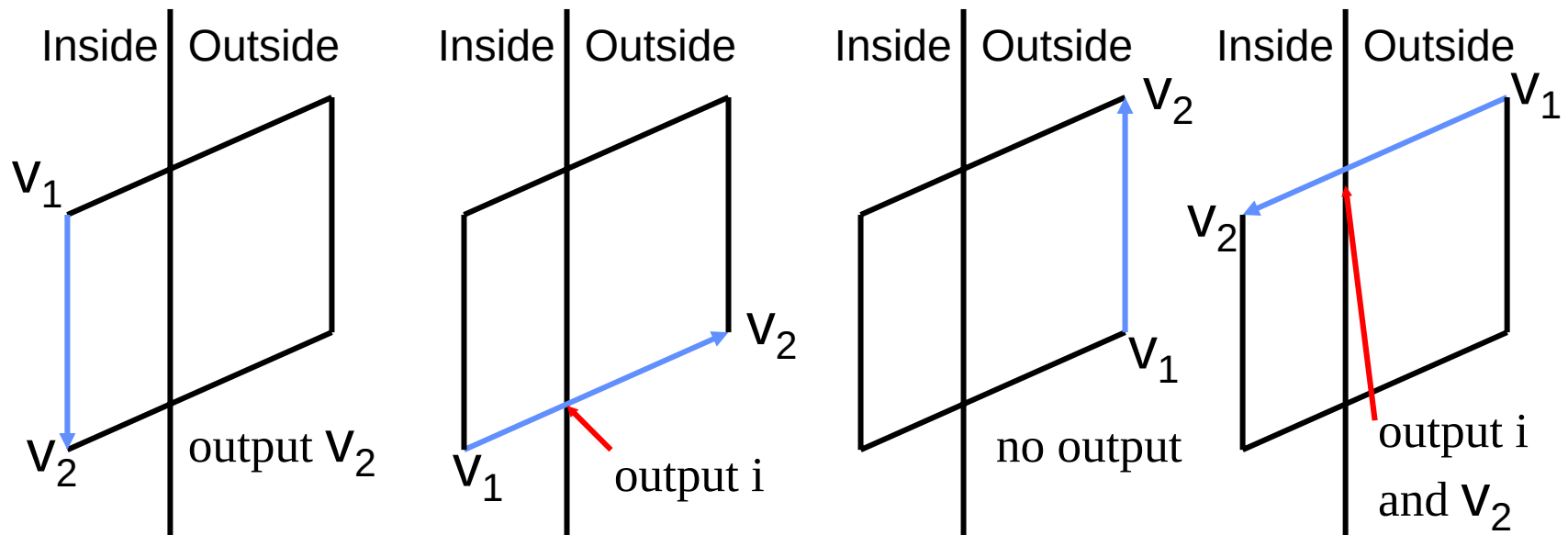
Sutherland-Hodgeman Polygon Clipping Algorithm

- Polygon clipping is similar to line clipping except we have to keep track of inside/outside relationships
 - Consider a polygon as a list of vertices
 - Note that clipping can increase the number of vertices!
 - Typically clip one edge at a time...



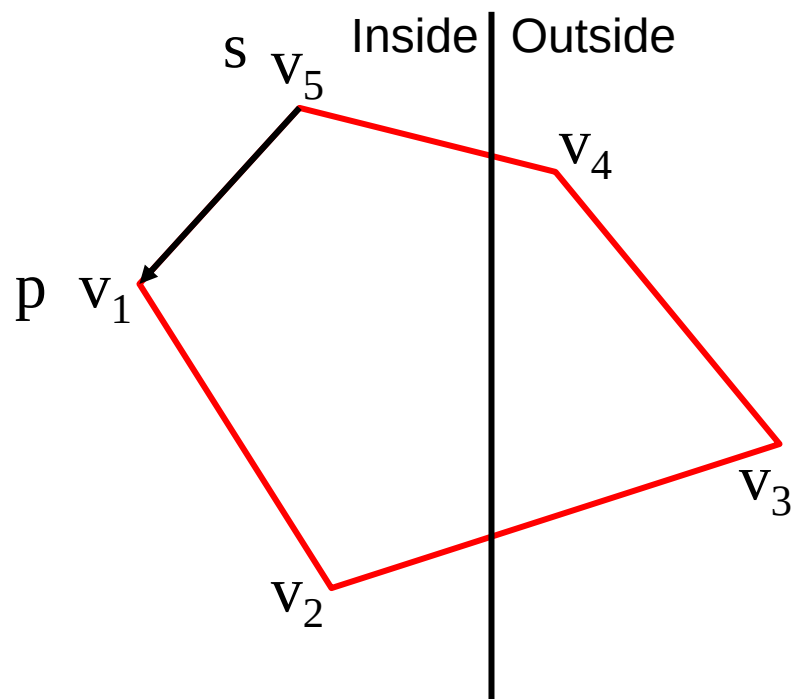
Sutherland-Hodgeman algorithm

- Present the vertices in pairs
 - $(v_n, v_1), (v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n)$
 - For each pair, what are the possibilities?
 - Consider v_1, v_2



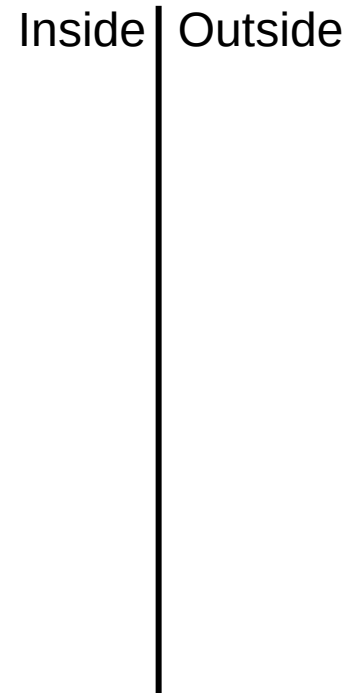
Example

V_5, V_1



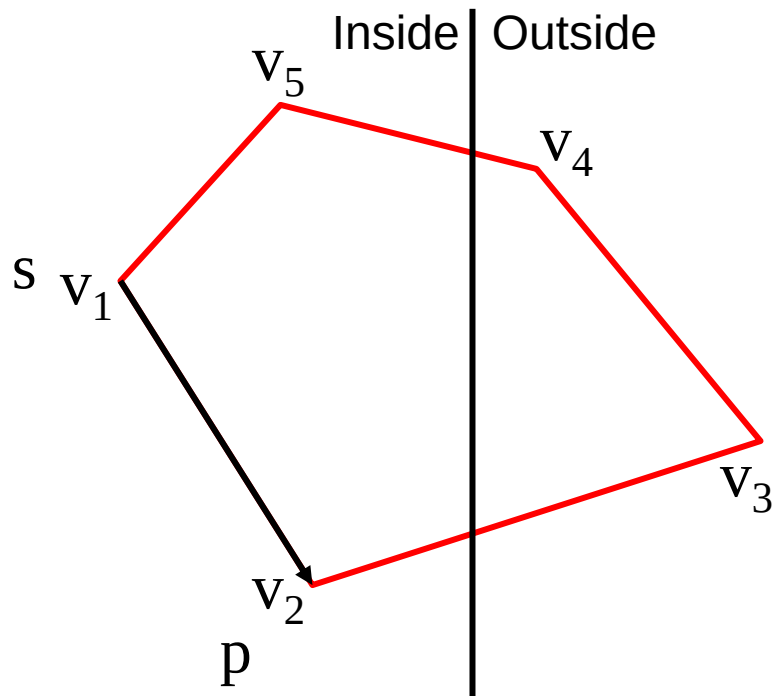
Inside, Inside
Output v_1

v_1^+

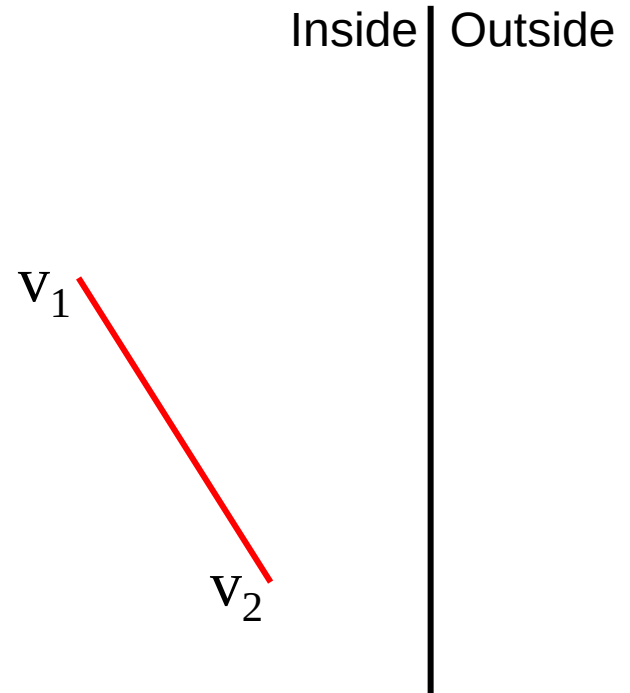


Current
Output

V_1, V_2

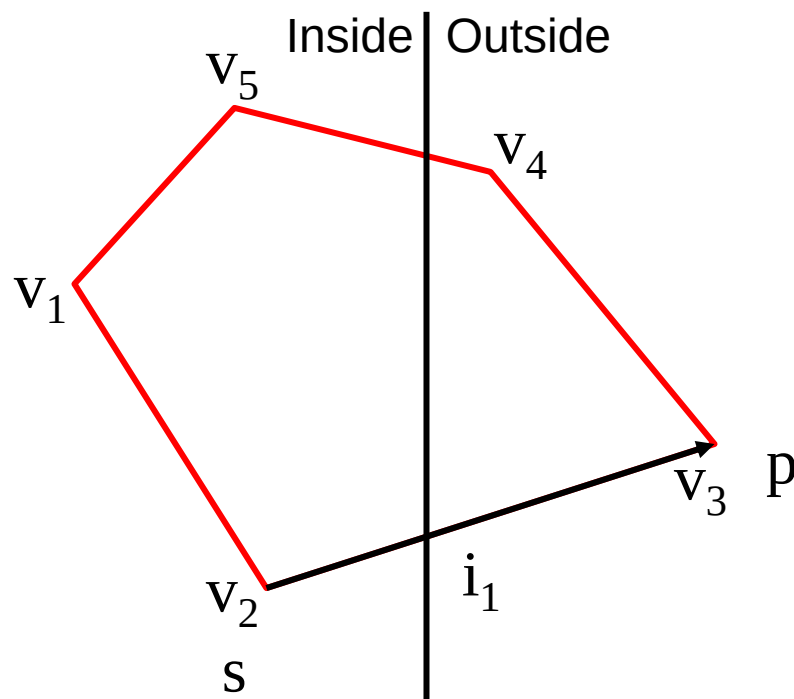


Inside, Inside
Output v_2

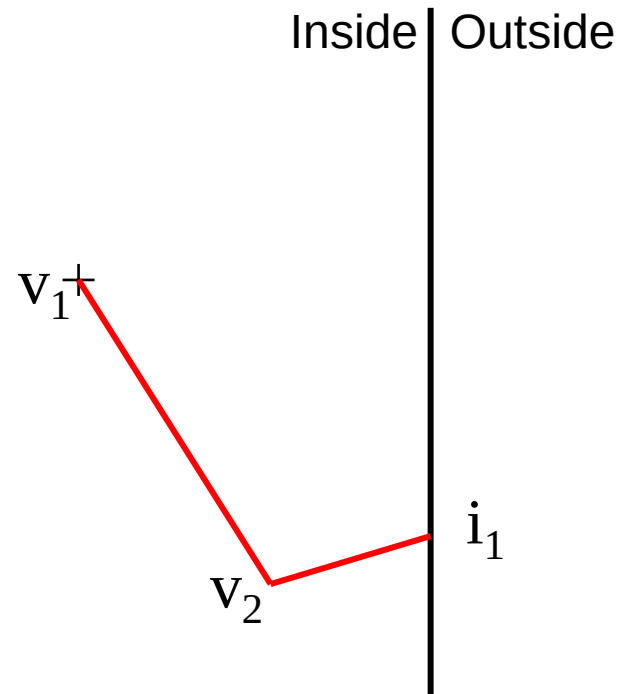


Current
Output

V_2, V_3

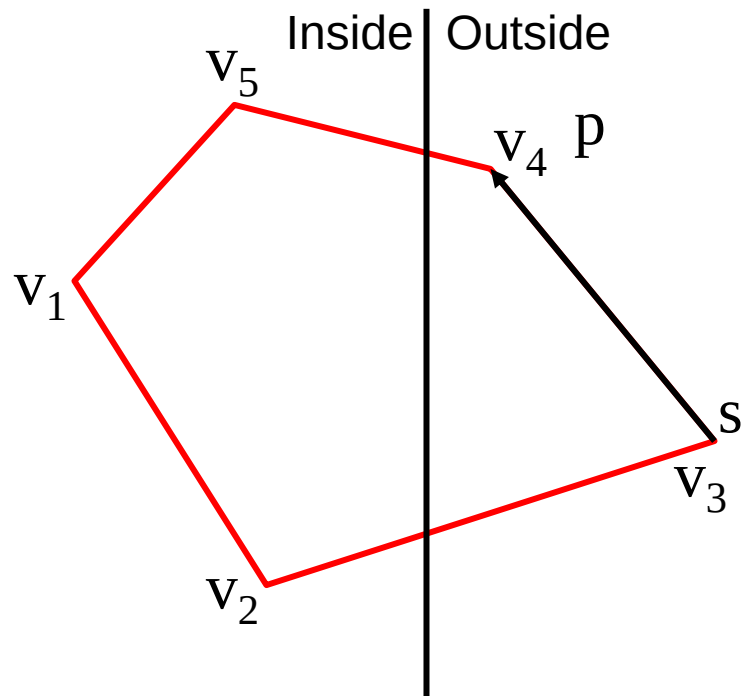


Inside, Outside
Output i_1

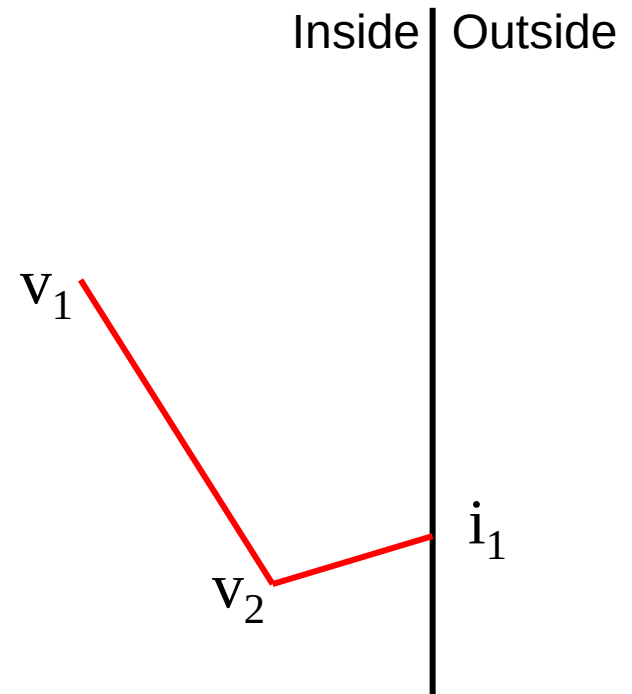


Current
Output

V_3, V_4

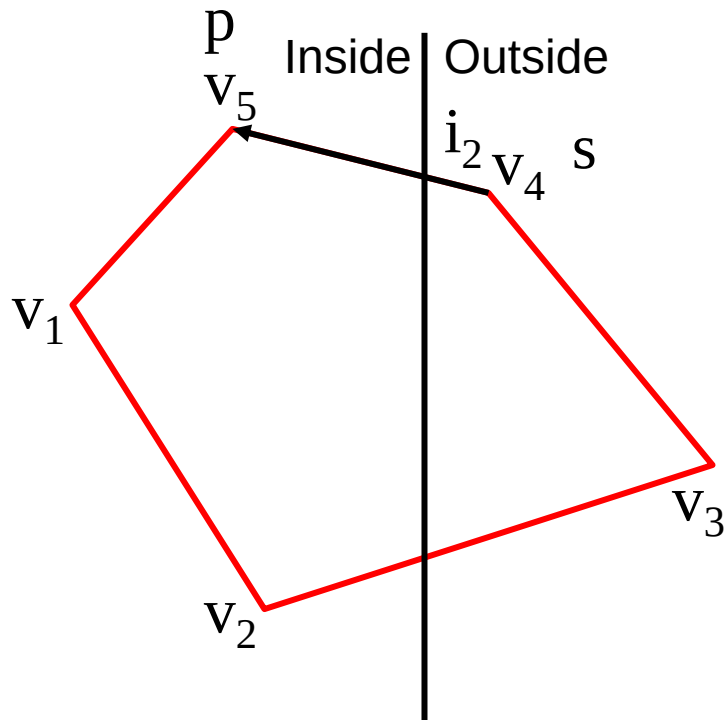


Outside, Outside
No output

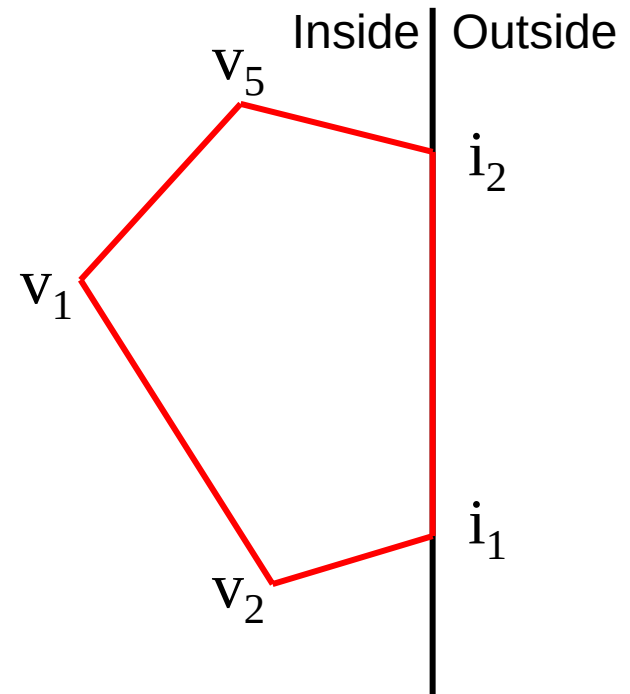


Current
Output

v_4, v_5 – last edge...



Outside, Inside
Output i_2, v_5



Current
Output

Thank You